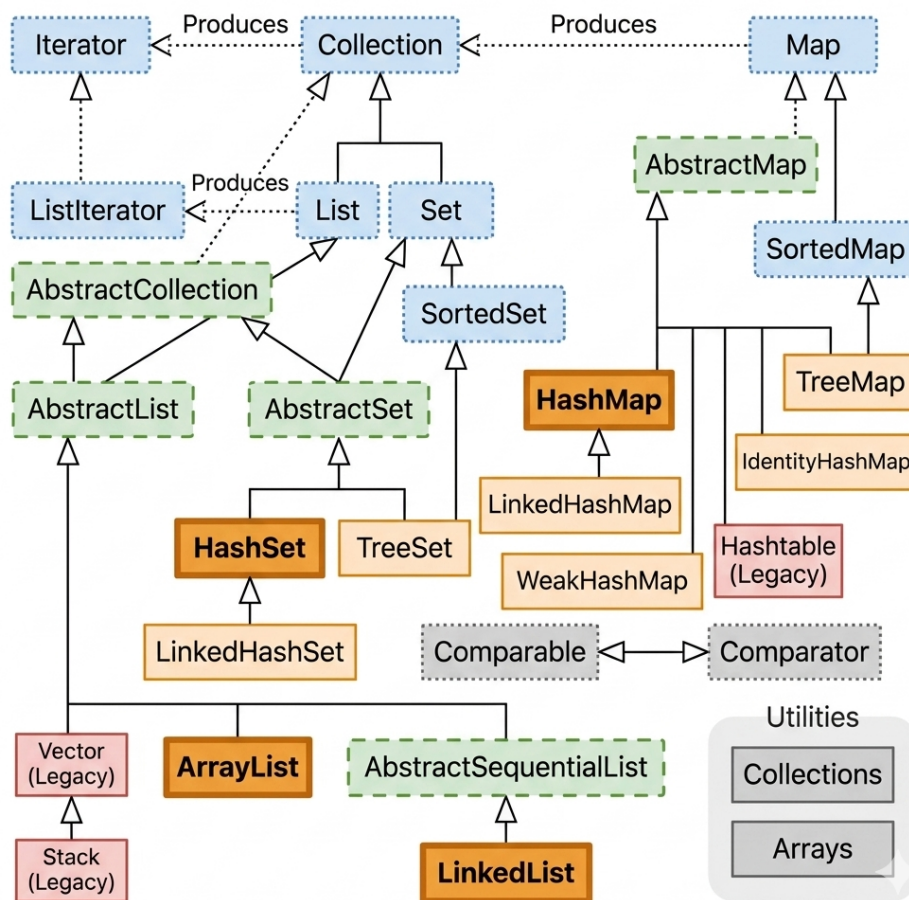


Java Collections Framework

Gerarchia, metodi distintivi e codice reale

Scheda di ripasso pronta da stampare



Idea chiave: dichiara le variabili usando l'interfaccia necessaria, e scegli la classe concreta in base a ordine, duplicati e prestazioni.

```
List<String> nomi = new ArrayList<>();
```

Ambiente dell'esame: OpenJDK 21 LTS. Strumenti consigliati: Visual Studio Code con Java Extension Pack, Git e Gradle.

Sono disponibili `record`, `Map.entry`, `Stream.toList()`, diamond operator, lambda, method reference e `var`; usa comunque `var` soltanto quando ammesso dallo stile del corso.

! Argomento ad alta frequenza negli appelli: quando compare questo simbolo, fermati e assicurati di saper scrivere il codice senza consultare la guida.

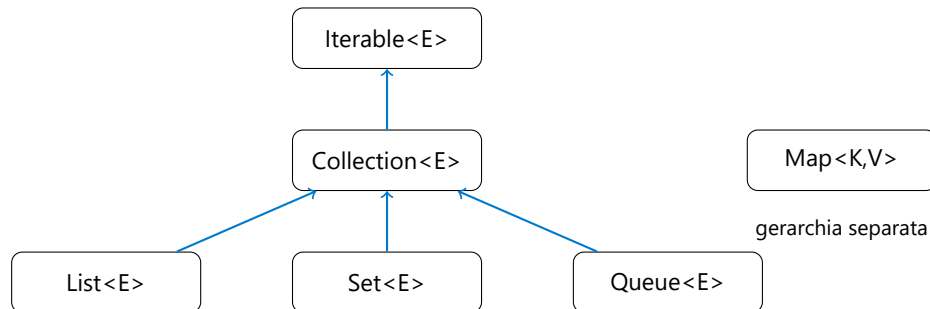
Indice

1	Come leggere la gerarchia	2
2	Iterable e Iterator	2
3	Collection: operazioni comuni	3
4	! List, ArrayList e LinkedList	3
4.1	List	3
4.2	ArrayList	4
4.3	LinkedList, ArrayDeque, Queue e Deque	5
5	! Set, HashSet, LinkedHashSet e TreeSet	5
5.1	Set	5
5.2	HashSet	6
5.3	LinkedHashSet	6
5.4	TreeSet	7
6	! Map, HashMap, LinkedHashMap e TreeMap	8
6.1	Map	8
6.2	! HashMap: conteggi e raggruppamenti	9
6.3	LinkedHashMap: ordine d'inserimento	9
6.4	TreeMap	10
7	Comparable e Comparator	11
8	Utility statiche: Collections e Arrays	11
8.1	Collections	12
8.2	Arrays	13
9	! Generics e functional interface	13
10	! Optional: risultati che possono mancare	15
11	! Stream, lambda e Collectors	15
11.1	Come funziona una pipeline Stream	15
11.2	! Operazioni terminali usate negli esami	16
11.3	flatMap e Collectors	17
12	Altre utility ricorrenti	18
12.1	Objects.requireNonNull	18
13	! Pattern bastardi degli esercizi OOP	18
13.1	! Factory, interfaccia e implementazione privata	19
13.2	Immutabilità e copie difensive	19
13.3	! Sequenze: indici e limiti	20
13.4	! Griglie: Position come chiave di Set e Map	21
14	Tabella di scelta rapida	21
15	! Errori tipici d'esame	22

1 Come leggere la gerarchia

Interfacce, classi astratte e classi concrete

- **Blu nell'immagine:** interfacce, cioè contratti che possono essere usati come tipi.
- **Verde:** classi astratte, cioè basi comuni per implementare nuove collezioni.
- **Arancione:** classi concrete, normalmente istanziate con `new`.
- **Map è separata da Collection:** gestisce associazioni chiave–valore, non una sequenza di elementi singoli.



2 Iterable e Iterator

Ruolo e comandi

`Iterable<E>` espone `iterator()` e può quindi essere usato in un `for-each`. L'oggetto `Iterator<E>` mantiene la posizione raggiunta durante la visita.

Comandi di Iterator: `hasNext()`, `next()`, `remove()` — se supportato dall'iteratore — e `forEachRemaining(action)`. Il metodo `iterator()` appartiene invece a `Iterable`; `forEach(action)` è disponibile direttamente su `Iterable`.

Visita manuale con Iterator

```

1 List<String> names = Arrays.asList("Anna", "Luca", "Sara");
2 Iterator<String> iterator = names.iterator();
3
4 // L'Iterator ricorda autonomamente la posizione corrente
5 while (iterator.hasNext()) {
6     String name = iterator.next();
7     System.out.println(name); // Anna, poi Luca, poi Sara
8 }
9
10 // Forma equivalente e più comune:
11 for (String name : names) {
12     System.out.println(name);
13 }
14
15 // Lambda: esegue println una volta per ogni elemento
16 names.forEach(name -> System.out.println(name));
17
18 // Riferimento a metodo: forma abbreviata equivalente
19 names.forEach(System.out::println);
20
21 // Sull'Iterator il nome corretto è forEachRemaining:
22 Iterator<String> remaining = names.iterator();
23 remaining.next(); // salta "Anna"
24 remaining.forEachRemaining(System.out::println); // Luca, Sara
  
```

Da ricordare

`Iterable` garantisce soltanto la possibilità di scorrere gli elementi: non possiede `size()` o `add()`. Questi metodi sono definiti da `Collection`.

3 Collection: operazioni comuni

Metodi usati nel corso

<code>add(e)</code> , <code>addAll(c)</code>	aggiungono uno o più elementi; restituiscono se la collezione è cambiata
<code>remove(e)</code> , <code>removeAll(c)</code>	rimuovono elementi; restituiscono se la collezione è cambiata
<code>retainAll(c)</code>	conserva soltanto gli elementi presenti anche in <code>c</code>
<code>contains(e)</code> , <code>containsAll(c)</code>	verificano la presenza di uno o più elementi
<code>size()</code> , <code>isEmpty()</code>	restituiscono la dimensione e indicano se la collezione è vuota
<code>clear()</code>	elimina tutti gli elementi
<code>toArray()</code> , <code>iterator()</code>	convertono in array o permettono la visita

Programmare verso l'interfaccia Collection

```

1 Collection<Integer> values = new ArrayList<>();
2 values.addAll(Arrays.asList(1, 3, 5, 7));
3
4 boolean added = values.add(9);    // true: 9 viene inserito
5 boolean removed = values.remove(3); // true: 3 era presente
6 boolean found = values.contains(5); // true
7
8 System.out.println(values);        // [1, 5, 7, 9]
9 System.out.println(values.size()); // 4
10
11 for (int value : values) {
12     System.out.println(value);
13 }

```

4 ! List, ArrayList e LinkedList

4.1 List

Metodi d'istanza distintivi di List

Una `List` ammette duplicati, conserva l'ordine degli elementi e permette l'accesso tramite indice. Il primo indice è zero.

<code>get(index)</code>	legge l'elemento in posizione <code>index</code>
<code>set(index, value)</code>	sostituisce e restituisce il vecchio elemento
<code>add(index, value)</code>	inserisce spostando a destra gli elementi successivi
<code>remove(index)</code>	rimuove per posizione e restituisce l'elemento rimosso
<code>remove(value)</code>	rimuove la prima occorrenza del valore e restituisce un booleano
<code>indexOf(value)</code>	prima posizione, oppure -1
<code>lastIndexOf(value)</code>	ultima posizione, oppure -1
<code>subList(from, to)</code>	vista dall'indice <code>from</code> incluso a <code>to</code> escluso
<code>listIterator()</code>	iteratore bidirezionale specifico delle liste

Operazioni distintive di List

```

1 List<String> list = new ArrayList<>(
2     Arrays.asList("A", "B", "A"))
3 ); // ordine e duplicati conservati
4
5 String second = list.get(1);    // "B"
6 String old = list.set(1, "X");  // old = "B"
7 list.add(1, "C");               // [A, C, X, A]
8 int firstA = list.indexOf("A"); // 0
9 int lastA = list.lastIndexOf("A"); // 3
10 List<String> part = list.subList(1, 3); // [C, X]
11 // part è una vista: le modifiche si riflettono anche su list

```

Attenzione a remove con List<Integer>

`numbers.remove(1)` interpreta 1 come indice.
Per rimuovere il valore intero 1 bisogna scrivere `numbers.remove(Integer.valueOf(1))`.

! Vista, copia o lista non modificabile?

`subList(from,to)` restituisce una **vista**: modifiche strutturali alla lista originale possono invalidarla.

`Arrays.asList(...)` ha dimensione fissa. `List.of(...)` e `List.copyOf(...)` non sono modificabili; inoltre rifiutano elementi null.

`new ArrayList<>(list.subList(from,to))` crea una copia modificabile;

`List.copyOf(list.subList(from,to))` crea invece una copia non modificabile. Questa distinzione viene spesso nascosta nei test.

Quattro liste che sembrano uguali, ma non lo sono

```

1 List<String> fixed = Arrays.asList("A", "B");
2 fixed.set(0, "X");           // consentito
3 // fixed.add("C");           // UnsupportedOperationException
4
5 List<String> immutable = List.of("A", "B");
6 // immutable.set(0, "X");    // UnsupportedOperationException
7
8 List<String> snapshot = List.copyOf(fixed);
9 // copia non modificabile dello stato corrente
10
11 List<String> mutableCopy = new ArrayList<>(fixed);
12 mutableCopy.add("C");        // consentito
13
14 List<String> source = new ArrayList<>(List.of("A", "B", "C"));
15 List<String> view = source.subList(0, 2); // vista collegata a source
16 List<String> editableCopy = new ArrayList<>(view);
17 List<String> readOnlyCopy = List.copyOf(view);

```

4.2 ArrayList**Quando sceglierla**

Array dinamico: `get(i)` e `set(i,e)` sono **O(1)**; `add(e)` in fondo è mediamente **O(1)**, mentre inserimenti e rimozioni nel mezzo sono **O(n)**. È la scelta iniziale più comune quando serve una lista.

Metodi propri: `ensureCapacity(n)` prepara spazio senza cambiare `size()`; `trimToSize()` riduce la capacità interna agli elementi presenti. Tutti gli altri comandi principali provengono da `List`.

ArrayList: capacità e accesso rapido

```

1 ArrayList<String> students = new ArrayList<>();
2 students.ensureCapacity(30); // capacità >= 30, ma size() == 0
3
4 students.add("Anna");
5 students.add("Luca");
6 students.add("Sara");
7
8 String first = students.get(0); // accesso O(1): "Anna"
9 students.remove(1);             // [Anna, Sara]
10 students.trimToSize();          // riduce solo lo spazio interno

```

4.3 LinkedList, ArrayDeque, Queue e Deque

Quando sceglierla

Lista doppiamente concatenata: l'accesso per indice è $O(n)$, mentre inserimenti e rimozioni alle estremità sono $O(1)$. Implementa sia `List` sia `Deque`.

Per una semplice coda o pila, `ArrayDeque` è normalmente la scelta preferita: è più compatta e rapida di `LinkedList` e non accetta `null`. Usa `LinkedList` quando serve davvero anche il contratto `List`.

Metodi d'istanza di Queue e Deque

<code>offer(e)</code>	prova a inserire in coda; restituisce <code>false</code> se non può
<code>poll()</code>	legge e rimuove la testa; <code>null</code> se vuota
<code>peek()</code>	legge la testa senza rimuoverla; <code>null</code> se vuota
<code>remove()</code> / <code>element()</code>	come <code>poll/peek</code> , ma lanciano eccezione se vuota
<code>addFirst(e)</code> , <code>addLast(e)</code>	inseriscono a una delle estremità
<code>pollFirst()</code> , <code>pollLast()</code>	rimuovono da un'estremità; <code>null</code> se vuota
<code>peekFirst()</code> , <code>peekLast()</code>	leggono un'estremità senza rimuovere
<code>push(e)</code> , <code>pop()</code>	uso LIFO: inseriscono e rimuovono dalla testa

LinkedList usata come coda FIFO

```

1 Queue<String> queue = new LinkedList<>();
2 queue.offer("A");
3 queue.offer("B");
4 queue.offer("C"); // ingresso: A, B, C
5
6 String first = queue.peek(); // "A", la coda non cambia
7 String out1 = queue.poll(); // esce "A"
8 String out2 = queue.poll(); // esce "B"
9 System.out.println(queue); // [C]
```

ArrayDeque usata come pila LIFO

```

1 Deque<String> deque = new ArrayDeque<>();
2 deque.addLast("B"); // [B]
3 deque.addLast("C"); // [B, C]
4 deque.push("A"); // [A, B, C]: equivale ad addFirst
5
6 String top = deque.pop(); // "A", resta [B, C]
7 String last = deque.removeLast(); // "C", resta [B]
```

5 ! Set, HashSet, LinkedHashSet e TreeSet

5.1 Set

Collezione senza duplicati

`Set` non aggiunge molti metodi a `Collection`, ma ne cambia il contratto: non ammette duplicati e non possiede indici. `HashSet` riconosce i duplicati tramite `equals/hashCode`; `LinkedHashSet` aggiunge un ordine di iterazione deterministico; `TreeSet` usa invece `compareTo` o il `Comparator`.

Metodi usati: `add`, `addAll`, `remove`, `contains`, `size`, `isEmpty`, `clear`, `iterator`. `add(e)` restituisce `false` quando l'elemento era già presente.

5.2 HashSet

Caratteristiche

Non garantisce alcun ordine di iterazione. `add`, `remove` e `contains` costano mediamente **O(1)**. Usa `hashCode()` per restringere la ricerca ed `equals()` per verificare l'uguaglianza.

! Chiavi ed elementi mutabili

Non modificare i campi usati da `equals()` o `hashCode()` mentre l'oggetto si trova in un `HashSet` o viene usato come chiave di `HashMap`. L'oggetto può diventare presente ma irraggiungibile tramite `contains` o `get`. Preferisci chiavi immutabili, ad esempio un record `Position(int x, int y)`.

HashSet: unicità e ricerca rapida

```
1 Set<String> tags = new HashSet<>();
2 boolean first = tags.add("java"); // true
3 boolean second = tags.add("oop"); // true
4 boolean duplicate = tags.add("java"); // false
5
6 System.out.println(tags.size()); // 2
7 System.out.println(tags.contains("oop")); // true
8 tags.remove("oop"); // resta soltanto "java"
9
10 // L'ordine di stampa non è garantito
11 System.out.println(tags);
```

5.3 LinkedHashSet

Unicità con ordine d'inserimento

`LinkedHashSet` mantiene gli elementi unici nell'ordine in cui vengono inseriti. Le operazioni principali restano mediamente **O(1)**. Reinserire con `add` un elemento già presente non ne cambia normalmente la posizione.

LinkedHashSet: eliminare duplicati senza perdere l'ordine

```
1 Set<String> names = new LinkedHashSet<>();
2 names.add("Anna");
3 names.add("Luca");
4 names.add("Anna"); // false: duplicato, l'ordine non cambia
5 names.add("Sara");
6
7 System.out.println(names); // [Anna, Luca, Sara]
```

5.4 TreeSet

Caratteristiche

Mantiene gli elementi ordinati secondo l'ordine naturale `Comparable` oppure secondo un `Comparator`. Le operazioni principali costano $O(\log n)$. Implementa `NavigableSet`.

Metodi d'istanza distintivi di TreeSet

<code>first()</code> , <code>last()</code>	minimo e massimo; eccezione se il set è vuoto
<code>lower(e)</code> , <code>higher(e)</code>	elemento strettamente minore o maggiore
<code>floor(e)</code> , <code>ceiling(e)</code>	minore/maggiore ammettendo anche l'uguaglianza
<code>pollFirst()</code> , <code>pollLast()</code>	rimuovono e restituiscono minimo o massimo; <code>null</code> se vuoto
<code>headSet(to)</code>	vista degli elementi minori di <code>to</code>
<code>tailSet(from)</code>	vista degli elementi maggiori o uguali a <code>from</code>
<code>subSet(from, to)</code>	vista da <code>from</code> incluso a <code>to</code> escluso
<code>descendingSet()</code>	vista con ordinamento inverso

TreeSet: ordine e ricerca per vicinanza

```

1 TreeSet<Integer> numbers = new TreeSet<>();
2 numbers.addAll(Arrays.asList(40, 10, 30, 20, 20));
3
4 System.out.println(numbers);    // [10, 20, 30, 40]
5 // Ordinato automaticamente e senza duplicati
6
7 System.out.println(numbers.lower(20)); // 10: strettamente <
8 System.out.println(numbers.floor(20)); // 20: <=
9 System.out.println(numbers.ceiling(25)); // 30: >=
10 System.out.println(numbers.higher(30)); // 40: strettamente >

```

6 ! Map, HashMap, LinkedHashMap e TreeMap

6.1 Map

Coppie chiave–valore

`Map<K,V>` non estende `Collection`: associa ogni chiave a un valore. Le chiavi sono uniche; una nuova chiamata a `put` con una chiave già presente sostituisce il valore precedente.

<code>put(key, value)</code>	inserisce e restituisce il vecchio valore
<code>putAll(map)</code>	copia tutte le associazioni
<code>get(key)</code>	valore associato; <code>null</code> se la chiave manca o è associata a <code>null</code>
<code>getOrDefault(key, def)</code>	valore associato, oppure <code>def</code> se la chiave manca
<code>putIfAbsent(key, value)</code>	inserisce se la chiave manca o è associata a <code>null</code>
<code>remove(key)</code>	elimina una chiave e restituisce il vecchio valore
<code>containsKey(key)</code>	verifica la presenza di una chiave
<code>containsValue(value)</code>	verifica la presenza di un valore
<code>compute(key, function)</code>	ricalcola il valore usando chiave e vecchio valore
<code>computeIfAbsent(key, function)</code>	calcola un valore se la chiave manca o vale <code>null</code>
<code>merge(key, value, function)</code>	inserisce il valore oppure lo combina con quello esistente
<code>keySet()</code>	vista <code>Set<K></code> delle chiavi
<code>values()</code>	vista <code>Collection<V></code> dei valori
<code>entrySet()</code>	vista delle coppie <code>Map.Entry<K,V></code>
<code>size(), isEmpty(), clear()</code>	numero di associazioni, controllo del vuoto e cancellazione

Se la funzione passata a `compute` restituisce `null`, l'associazione viene rimossa oppure non viene creata.

Map: inserimento, accesso e viste

```

1 Map<String, Integer> ages = new HashMap<>();
2 ages.put("Anna", 20);
3 ages.put("Luca", 25);
4 Integer oldAge = ages.put("Anna", 21); // restituisce 20
5
6 int anna = ages.get("Anna");           // 21
7 int sara = ages.getOrDefault("Sara", 0); // 0
8 boolean known = ages.containsKey("Luca"); // true
9 // containsKey distingue una chiave assente da una chiave associata a null
10
11 for (Map.Entry<String, Integer> entry : ages.entrySet()) {
12     // entry rappresenta una coppia chiave-valore
13     System.out.println(entry.getKey() + " -> " + entry.getValue());
14 }
```

! get non distingue assente da presente con null

`map.get(key) == null` può significare sia "chiave assente" sia "chiave presente e associata a `null`". Quando la distinzione conta usa `containsKey(key)`. Negli esercizi d'esame, se `null` non ha un significato richiesto, evitarlo rende la logica molto più sicura.

6.2 ! HashMap: conteggi e raggruppamenti

Caratteristiche

Non garantisce l'ordine delle chiavi. `get`, `put`, `remove` e `containsKey` costano mediamente **$O(1)$** . Applica `hashCode()` ed `equals()` alle chiavi. È la scelta iniziale più comune quando serve una mappa. Non aggiunge comandi fondamentali a `Map`: cambia soprattutto l'implementazione e le prestazioni.

Conteggio: il template più rapido da ricordare

```
1 Map<String, Integer> frequency = new HashMap<>();
2 List<String> words = Arrays.asList("java", "oop", "java", "java");
3
4 for (String word : words) {
5     frequency.put(word, frequency.getOrDefault(word, 0) + 1);
6 }
7
8 System.out.println(frequency.get("java")); // 3
9 System.out.println(frequency); // ordine non garantito
```

Alternative utili: `putIfAbsent`, `computeIfAbsent` e `merge`

```
1 Map<String, List<Integer>> grades = new HashMap<>();
2
3 grades.putIfAbsent("Anna", new ArrayList<>());
4 // Inserisce la lista se "Anna" manca oppure è associata a null
5
6 grades.computeIfAbsent("Luca", ignored -> new ArrayList<>()).add(28);
7 // Se "Luca" manca, crea la lista e poi aggiunge 28
8
9 Map<String, Integer> points = new HashMap<>();
10 points.merge("Anna", 3, (oldValue, value) -> oldValue + value);
11 points.merge("Anna", 1, Integer::sum);
12 // Prima chiamata: Anna=3. Seconda chiamata: Anna=4
```

6.3 LinkedHashMap: ordine d'inserimento

Quando serve

`LinkedHashMap` mantiene normalmente l'ordine di inserimento delle chiavi, con operazioni medie **$O(1)$** . È la scelta giusta quando il test confronta anche l'ordine del risultato, per esempio in un motore di regole.

Regole applicate nello stesso ordine in cui sono aggiunte

```
1 Map<String, Predicate<Integer>> rules = new LinkedHashMap<>();
2 rules.put("positivo", value -> value > 0);
3 rules.put("pari", value -> value % 2 == 0);
4
5 List<String> matched = new ArrayList<>();
6 for (Map.Entry<String, Predicate<Integer>> entry : rules.entrySet()) {
7     if (entry.getValue().test(4)) {
8         matched.add(entry.getKey());
9     }
10 }
11 // [positivo, pari]: ordine deterministico
```

6.4 TreeMap

Caratteristiche

Mantiene le chiavi ordinate. Le operazioni principali costano $O(\log n)$ e richiedono chiavi `Comparable` oppure un `Comparator`. I metodi di navigazione appartengono a `NavigableMap`: per invocarli, il tipo della variabile deve essere `NavigableMap` o `TreeMap`, non soltanto `Map`.

Metodi d'istanza distintivi di TreeMap

<code>firstKey()</code> , <code>lastKey()</code>	prima e ultima chiave
<code>lowerKey(k)</code> , <code>higherKey(k)</code>	chiave strettamente minore o maggiore
<code>floorKey(k)</code> , <code>ceilingKey(k)</code>	chiave vicina ammettendo l'uguaglianza
<code>firstEntry()</code> , <code>lastEntry()</code>	prima e ultima coppia chiave-valore
<code>pollFirstEntry()</code> , <code>pollLastEntry()</code>	restituiscono e rimuovono una coppia estrema
<code>headMap(k)</code> , <code>tailMap(k)</code>	viste delle chiavi minori di <code>k</code> o maggiori/uguali a <code>k</code>
<code>subMap(from, to)</code>	vista da <code>from</code> incluso a <code>to</code> escluso
<code>descendingMap()</code>	vista con chiavi in ordine inverso

TreeMap: chiavi automaticamente ordinate

```

1 Map<String, Integer> scores = new TreeMap<>();
2 scores.put("Zoe", 28);
3 scores.put("Anna", 30);
4 scores.put("Luca", 25);
5 scores.put("Anna", 31); // sostituisce il voto precedente
6
7 System.out.println(scores);
8 // {Anna=31, Luca=25, Zoe=28}: ordine naturale delle String

```

7 Comparable e Comparator

Due modi di definire l'ordine

Comparable<T>: definisce dentro la classe un unico ordine naturale tramite `compareTo`.

Comparator<T>: definisce all'esterno un criterio di ordinamento; per lo stesso tipo se ne possono creare diversi.

Il confronto restituisce un numero negativo se il primo elemento viene prima, zero se i due elementi sono equivalenti per l'ordinamento e un numero positivo se il primo viene dopo.

`TreeSet` e `TreeMap` considerano duplicati due elementi o chiavi quando il confronto restituisce zero, anche se `equals()` restituisce `false`.

Comparable: ordine naturale dentro Person

```
1 class Person implements Comparable<Person> {
2     private final String name;
3     private final int birthYear;
4
5     Person(String name, int birthYear) {
6         this.name = name;
7         this.birthYear = birthYear;
8     }
9
10    public String getName() {
11        return this.name;
12    }
13
14    @Override
15    public int compareTo(Person other) {
16        return Integer.compare(this.birthYear, other.birthYear);
17    }
18 }
```

Comparator: ordine esterno alternativo

```
1 Comparator<Person> byName = new Comparator<>() {
2     @Override
3     public int compare(Person first, Person second) {
4         return first.getName().compareTo(second.getName());
5     }
6 };
7
8 Set<Person> ordered = new TreeSet<>(byName);
```

8 Utility statiche: Collections e Arrays

Attenzione: statici oppure d'istanza?

`list.add()`, `set.contains()` e `map.put()` sono metodi **d'istanza**: operano su uno specifico oggetto.

`Collections.sort(list)` e `Arrays.sort(array)` sono invece metodi **statici**: appartengono alle classi di utilità e ricevono la struttura su cui lavorare come argomento.

8.1 Collections

Metodi statici principali

<code>sort(list)</code>	ordina secondo l'ordine naturale
<code>sort(list, comparator)</code>	ordina usando un criterio esterno
<code>binarySearch(list, key)</code>	indice trovato, altrimenti $-(\text{insertionPoint}) - 1$
<code>reverse(list)</code>	inverte l'ordine degli elementi
<code>shuffle(list)</code>	permuta casualmente gli elementi
<code>rotate(list, distance)</code>	ruota gli elementi di alcune posizioni
<code>swap(list, i, j)</code>	scambia due posizioni
<code>fill(list, value)</code>	sostituisce ogni elemento con lo stesso valore
<code>copy(dest, source)</code>	sovrascrive <code>dest</code> , che deve avere già almeno <code>source.size()</code> elementi
<code>replaceAll(list, old, new)</code>	sostituisce tutte le occorrenze
<code>min(collection)</code> , <code>max(collection)</code>	trova minimo e massimo
<code>frequency(collection, value)</code>	conta quante volte compare un valore
<code>disjoint(c1, c2)</code>	verifica che non esistano elementi comuni
<code>nCopies(n, value)</code>	restituisce una lista non modificabile con n copie

! Regola esatta di `binarySearch`

La lista o l'array devono essere già ordinati con lo stesso criterio usato dalla ricerca. Se la chiave manca, il risultato è $-(\text{insertionPoint}) - 1$, dove `insertionPoint` è la posizione in cui andrebbe inserita mantenendo l'ordinamento.

Collections: ordinamento e ricerca

```

1 List<Integer> list = new ArrayList<>(List.of(10, 20, 40));
2
3 // La lista deve essere già ordinata con lo stesso criterio
4 int position = Collections.binarySearch(list, 30);
5 // insertionPoint = 2, quindi risultato = -2 - 1 = -3
6
7 Collections.reverse(list); // [40, 20, 10]
8 Collections.swap(list, 0, 1); // [20, 40, 10]
9 Collections.shuffle(list); // nuovo ordine casuale

```

Collections: interrogazioni e modifiche

```

1 List<Integer> values = new ArrayList<>(
2     Arrays.asList(3, 1, 3, 2)
3 );
4
5 int max = Collections.max(values); // 3
6 int times = Collections.frequency(values, 3); // 2
7 boolean separate = Collections.disjoint(
8     values, Arrays.asList(8, 9)
9 ); // true: nessun elemento comune
10
11 Collections.replaceAll(values, 3, 7); // [7, 1, 7, 2]
12 List<String> copies = Collections.nCopies(3, "Java");
13 // copies = [Java, Java, Java], lista non modificabile

```

8.2 Arrays

Metodi statici principali

<code>sort(array)</code>	ordina tutto l'array
<code>sort(array, from, to)</code>	ordina da from incluso a to escluso
<code>binarySearch(array, key)</code>	indice trovato, altrimenti $-(\text{insertionPoint}) - 1$
<code>fill(array, value)</code>	riempie tutte le celle con un valore
<code>fill(array, from, to, value)</code>	riempie da from incluso a to escluso
<code>copyOf(array, length)</code>	crea una copia con nuova lunghezza
<code>copyOfRange(array, from, to)</code>	copia da from incluso a to escluso
<code>equals(a, b)</code>	confronta due array elemento per elemento
<code>deepEquals(a, b)</code>	confronto ricorsivo di array annidati
<code>toString(array)</code>	rappresentazione leggibile di un array
<code>deepToString(array)</code>	rappresentazione di array multidimensionali
<code>asList(elements)</code>	restituisce una lista a dimensione fissa collegata all'array

Arrays: ordinamento, ricerca e riempimento

```

1 int[] values = {4, 1, 3, 2};
2 Arrays.sort(values); // modifica lo stesso array
3 System.out.println(Arrays.toString(values));
4 // [1, 2, 3, 4]
5
6 int position = Arrays.binarySearch(values, 3); // indice 2
7 Arrays.fill(values, 1, 3, 9);
8 // [1, 9, 9, 4]: intervallo [from, to)

```

Arrays: copie, confronti e conversioni

```

1 int[] original = {1, 2, 3, 4};
2 int[] copy = Arrays.copyOf(original, 6);
3 // [1, 2, 3, 4, 0, 0]: celle nuove al valore di default
4
5 int[] part = Arrays.copyOfRange(original, 1, 3);
6 // [2, 3]: indice iniziale incluso, finale escluso
7
8 boolean same = Arrays.equals(
9     original, new int[]{1, 2, 3, 4}
10 ); // true: confronta il contenuto
11
12 int[][] matrix = {{1, 2}, {3, 4}};
13 System.out.println(Arrays.deepToString(matrix)); // [[1, 2], [3, 4]]
14
15 List<String> fixed = Arrays.asList("A", "B", "C");
16 fixed.set(0, "X"); // consentito: [X, B, C]
17 // fixed.add("D"); // UnsupportedOperationException: size fissa
18
19 int[] primitive = {1, 2, 3};
20 List<int> oneElement = Arrays.asList(primitive);
21 // Con un array primitivo crea una lista di size 1 contenente tutto l'array

```

9 ! Generics e functional interface

T, Predicate, Function e BiFunction

T è un parametro di tipo: il codice non deve presumere che sia `String`, `Integer` o un altro tipo concreto.

<code>Predicate<T></code>	riceve un T e restituisce <code>boolean</code> ; metodo <code>test</code>
<code>Function<T, R></code>	trasforma un T in un R; metodo <code>apply</code>
<code>BiFunction<T, U, R></code>	riceve due valori e produce un R; metodo <code>apply</code>

Selezionare elementi con un Predicate<T>

```

1 public static <T> List<T> select(
2     List<T> values,
3     Predicate<T> predicate) {
4     List<T> result = new ArrayList<>();
5     for (T value : values) {
6         if (predicate.test(value)) {
7             result.add(value);
8         }
9     }
10    return result;
11 }
12
13 List<Integer> positives = select(
14     List.of(-2, 3, 0, 5),
15     value -> value > 0
16 );

```

! Non inventare operazioni su T

Su un valore di tipo T puoi usare soltanto le operazioni garantite dalla firma. Non puoi sommare, ordinare o chiamare metodi specifici senza un vincolo o senza ricevere un **Comparator**, una **Function** o un'altra strategia dall'esterno.

Method reference: cosa significa Tipo::metodo

Un method reference è una forma abbreviata di lambda. I parametri non sono scritti esplicitamente: vengono ricavati dall'interfaccia funzionale richiesta in quel punto.

<code>Integer::compare</code>	equivale a <code>(a, b) -> Integer.compare(a, b)</code>
<code>System.out::println</code>	equivale a <code>value -> System.out.println(value)</code>
<code>Map.Entry::getKey</code>	equivale a <code>entry -> entry.getKey()</code>
<code>ArrayList::new</code>	equivale a <code>() -> new ArrayList<>()</code>

`Integer::compare` richiama il metodo statico `Integer.compare(int, int)`: riceve due interi e restituisce un numero negativo, zero o positivo. Può quindi essere usato dove serve un `Comparator<Integer>`.

Lambda e method reference equivalenti

```

1 Comparator<Integer> first =
2     (a, b) -> Integer.compare(a, b);
3
4 Comparator<Integer> second = Integer::compare;
5
6 Optional<Integer> minimum = List.of(7, 3, 10)
7     .stream()
8     .min(Integer::compare); // Optional[3]

```

Combinare due liste generiche: zip

```

1 record Pair<X, Y>(X first, Y second) { }
2
3 public static <X, Y> List<Pair<X, Y>> zip(
4     List<X> xs,
5     List<Y> ys) {
6     List<Pair<X, Y>> result = new ArrayList<>();
7     int size = Math.min(xs.size(), ys.size());
8     for (int i = 0; i < size; i++) {
9         result.add(new Pair<>(xs.get(i), ys.get(i)));
10    }
11    return result;
12 }

```

! Map.entry in OpenJDK 21

`Map.entry(key, value)` crea rapidamente una coppia non modificabile, ma rifiuta sia chiavi sia valori `null`. Per una coppia generica che deve accettare `null`, un piccolo `record Pair<X, Y>` è più adatto.

10 ! Optional: risultati che possono mancare

Assenza esplicita, non null nascosto

`Optional<T>` rappresenta un risultato presente oppure assente. `Optional.of(value)` richiede un valore non nullo; `Optional.ofNullable(value)` accetta anche `null`; `Optional.empty()` rappresenta l'assenza.

Ricerca del primo elemento: template d'esame

```
1 public static <T> Optional<T> firstMatching(
2     List<T> values,
3     Predicate<T> condition) {
4     for (T value : values) {
5         if (condition.test(value)) {
6             return Optional.of(value);
7         }
8     }
9     return Optional.empty();
10 }
```

! `orElse` e `orElseGet` non sono equivalenti

`optional.orElse(expensive())` valuta `expensive()` anche quando il valore è presente. `optional.orElseGet(() -> expensive())` esegue invece il calcolo soltanto se l'`Optional` è vuoto. Evita `get()` senza controllo.

11 ! Stream, lambda e Collectors

11.1 Come funziona una pipeline Stream

Sorgente, operazioni intermedie e operazione terminale

Uno `Stream<T>` non è una collezione e non contiene direttamente gli elementi: rappresenta una pipeline che li elabora a partire da una sorgente.

Lambda: una scrittura come `name -> name.length()` rappresenta una piccola funzione senza nome. A sinistra di `->` ci sono i parametri; a destra c'è l'operazione da eseguire o il valore da restituire.

Import da ricordare:

`collection.stream()` può essere chiamato senza importare `Stream`.

`Collectors.toList()` richiede `import java.util.stream.Collectors;`.

Se si dichiara una variabile `Stream<T>`, serve anche `import java.util.stream.Stream;`.

`import java.util.*;` non importa il sottopackage `java.util.stream`.

! `toList()` oppure `Collectors.toList()`?

`stream.toList()` è disponibile da Java 16 e restituisce una lista non modificabile.

`stream.collect(Collectors.toList())` funziona anche nelle versioni precedenti, ma non garantisce il tipo concreto né formalmente la modificabilità. Se serve sicuramente una `ArrayList` modificabile usa `collect(Collectors.toCollection(ArrayList::new))`.

<code>collection.stream()</code>	crea lo stream dalla collezione
<code>filter(predicate)</code>	conserva gli elementi che soddisfano una condizione
<code>map(function)</code>	trasforma ogni elemento in un altro valore
<code>flatMap(function)</code>	trasforma ogni elemento in uno stream e poi li appiattisce
<code>mapToInt(function)</code>	produce valori <code>int</code> , permettendo <code>sum()</code>
<code>sorted()</code>	ordina secondo l'ordine naturale; accetta anche un <code>comparator</code>
<code>distinct()</code>	elimina i duplicati secondo <code>equals()</code>

Intermedie oppure terminali

`filter`, `map` e `flatMap` sono operazioni **intermedie**: costruiscono la pipeline e restituiscono un altro `Stream`, ma non elaborano gli elementi finché non arriva un'operazione terminale. `collect`, `count`, `sum` e `forEach` sono **terminali**: producono il risultato e consumano lo `Stream`. Uno `Stream` consumato non può essere riutilizzato.

Stream: filtrare, trasformare e raccogliere

```

1 List<String> names = Arrays.asList("Anna", "Luca", "Alberto", "Sara");
2
3 List<String> result = names.stream() // sorgente
4     .filter(name -> name.startsWith("A")) // solo nomi con A
5     .map(name -> name.toUpperCase()) // trasforma ogni nome
6     .collect(Collectors.toList()); // operazione terminale
7
8 System.out.println(result); // [ANNA, ALBERTO]
9 System.out.println(names); // la lista originale non cambia

```

11.2 ! Operazioni terminali usate negli esami**Produrre un risultato**

<code>forEach(action)</code>	esegue un'azione per ogni elemento e non restituisce un valore
<code>collect(collector)</code>	raccoglie il risultato, ad esempio in una lista, set, mappa o stringa
<code>count()</code>	restituisce il numero di elementi come <code>long</code>
<code>min(comparator),</code> <code>max(comparator)</code>	restituiscono un <code>Optional<T></code>
<code>sum()</code>	somma uno stream numerico, ad esempio <code>IntStream</code>
<code>anyMatch(predicate)</code>	vero se almeno un elemento soddisfa la condizione
<code>allMatch(predicate)</code>	vero se tutti soddisfano la condizione
<code>findFirst(), findAny()</code>	restituiscono un elemento dentro un <code>Optional<T></code>

Stream numerico: mapToInt, sum e count

```

1 List<Integer> grades = Arrays.asList(18, 24, 30, 27);
2
3 int total = grades.stream()
4     .mapToInt(value -> value) // Stream<Integer> diventa IntStream
5     .sum(); // 99
6
7 long passed = grades.stream()
8     .filter(value -> value >= 18)
9     .count(); // 4: count restituisce long
10
11 boolean allPassed = grades.stream()
12     .allMatch(value -> value >= 18); // true

```

Optional: risultato che potrebbe non esistere

```

1 List<Integer> empty = new ArrayList<>();
2
3 int minimum = empty.stream()
4     .min(Integer::compare)
5     .orElse(-1);
6 // Lo stream è vuoto: min restituisce Optional.empty()
7 // orElse(-1) fornisce un valore alternativo sicuro
8
9 int found = Arrays.asList(3, 7, 10).stream()
10     .filter(value -> value % 2 == 0)
11     .findAny()
12     .orElse(-1); // 10, oppure -1 se nessun numero pari esiste

```

11.3 flatMap e Collectors

Metodi di Collectors più usati

<code>Collectors.toList()</code>	raccoglie gli elementi in una lista; il tipo concreto non è garantito
<code>Collectors.toSet()</code>	raccoglie in un set; tipo concreto e ordine non sono garantiti
<code>Collectors.toMap(k, v)</code>	crea una mappa; chiavi duplicate causano un'eccezione
<code>Collectors.joining(sep)</code>	concatena stringhe usando un separatore

! toMap e chiavi duplicate

Se due elementi producono la stessa chiave, la forma a due argomenti di `Collectors.toMap` lancia `IllegalStateException`. Fornisci una funzione di fusione quando i duplicati sono possibili.

toMap con chiavi duplicate: funzione di fusione

```
1 Map<String, Integer> totals = Stream.of(
2     Map.entry("Anna", 3),
3     Map.entry("Anna", 1),
4     Map.entry("Luca", 2)
5 ).collect(Collectors.toMap(
6     Map.Entry::getKey,
7     Map.Entry::getValue,
8     Integer::sum
9 ));
10 // {Anna=4, Luca=2}: ordine non garantito
```

flatMap: appiattare collezioni annidate

```
1 List<List<String>> groups = Arrays.asList(
2     Arrays.asList("Anna", "Luca"),
3     Arrays.asList("Sara", "Anna")
4 );
5
6 Set<String> uniqueNames = groups.stream()
7     .flatMap(group -> group.stream())
8     // Da Stream<List<String>> a un solo Stream<String>
9     .collect(Collectors.toSet());
10
11 System.out.println(uniqueNames); // ordine non garantito, senza duplicati
```

Collectors: creare Map e String

```
1 List<String> names = Arrays.asList("Anna", "Luca", "Sara");
2
3 Map<String, Integer> lengths = names.stream()
4     .collect(Collectors.toMap(
5         name -> name,           // chiave: il nome
6         name -> name.length()   // valore: la lunghezza
7     ));
8 // {Anna=4, Luca=4, Sara=4}: ordine non garantito
9
10 String text = names.stream()
11     .collect(Collectors.joining(", "));
12 // "Anna, Luca, Sara"
```

12 Altre utility ricorrenti

12.1 Objects.requireNonNull

Controllare subito gli argomenti

`Objects.requireNonNull(value)` è un metodo statico di `Objects`. Se il valore è `null`, lancia immediatamente `NullPointerException`; altrimenti restituisce il valore ricevuto. È utile nei costruttori per rifiutare subito argomenti non validi.

Validazione nel costruttore

```
1 class Student {
2     private final String name;
3
4     Student(String name) {
5         this.name = Objects.requireNonNull(
6             name, "Il nome non può essere null"
7         );
8         // Errore immediato nel punto in cui nasce l'oggetto
9     }
10 }
```

13 ! Pattern bastardi degli esercizi OOP

! Prima i test, poi l'implementazione

Quando sono presenti test JUnit, trattali come specifica eseguibile: controlla firme, ordine dei risultati, mutabilità e casi limite. Implementa prima il comportamento minimo osservato dai test; non costruire un sistema più grande di quello richiesto.

Ordine operativo consigliato

1. Apri `Test.java` e annota i metodi realmente verificati.
2. Leggi le interfacce e rispetta esattamente firme e tipi generici.
3. Scegli `List`, `Set` o `Map` in base al significato dello stato.
4. Implementa prima la versione più semplice con cicli e collection.
5. Verifica ordine, duplicati, assenza del risultato e casi limite.
6. Per gli esercizi grafici concentra il lavoro nella logica; modifica la GUI soltanto se necessario.

Priorità di ripasso

1. Collections e Map
2. lettura dei test
3. factory e classi Impl
4. generics e Optional
5. lambda e Predicate
6. logica su griglie.

13.1 ! Factory, interfaccia e implementazione privata

Forma ricorrente dell'Esercizio 1

La factory restituisce il tipo dell'interfaccia, mentre la classe concreta può restare privata. Il chiamante conosce il contratto, non i dettagli dello stato interno.

Factory + Map per un contatore

```

1 interface Counter {
2     void hit(String key);
3     int count(String key);
4 }
5
6 interface CounterFactory {
7     Counter empty();
8 }
9
10 class CounterFactoryImpl implements CounterFactory {
11     @Override
12     public Counter empty() {
13         return new CounterImpl();
14     }
15
16     private static class CounterImpl implements Counter {
17         private final Map<String, Integer> counts = new HashMap<>();
18
19         @Override
20         public void hit(String key) {
21             counts.put(key, counts.getOrDefault(key, 0) + 1);
22         }
23
24         @Override
25         public int count(String key) {
26             return counts.getOrDefault(key, 0);
27         }
28     }
29 }

```

13.2 Immutabilità e copie difensive

Restituire un nuovo oggetto

Se i test conservano una vecchia versione e verificano che non cambi, ogni operazione deve creare una nuova collezione. `List.copyOf` è utile per salvare o restituire uno snapshot non modificabile.

Builder persistente: add non modifica this

```

1 final class ListBuilder<T> {
2     private final List<T> elements;
3
4     ListBuilder() {
5         this.elements = List.of();
6     }
7
8     private ListBuilder(List<T> elements) {
9         this.elements = List.copyOf(elements);
10    }
11
12    ListBuilder<T> add(T element) {
13        List<T> copy = new ArrayList<>(this.elements);
14        copy.add(element);
15        return new ListBuilder<>(copy);
16    }
17
18    List<T> build() {
19        return this.elements;
20    }
21 }

```

13.3 ! Sequenze: indici e limiti

! Il caso limite rompe il codice elegante

Per finestre di lunghezza `size`, l'ultimo indice iniziale valido soddisfa `i + size <= list.size()`. Decidi esplicitamente cosa fare con `size <= 0`; in genere si lancia `IllegalArgumentException`. Con due liste di lunghezza diversa, `zip` usa normalmente la lunghezza minore, salvo diversa indicazione dei test.

Finestre sovrapposte con copia della subList

```
1 public static <T> List<List<T>> windows(  
2     List<T> list,  
3     int size) {  
4     if (size <= 0) {  
5         throw new IllegalArgumentException("size must be positive");  
6     }  
7  
8     List<List<T>> result = new ArrayList<>();  
9     for (int i = 0; i + size <= list.size(); i++) {  
10         result.add(List.copyOf(list.subList(i, i + size)));  
11     }  
12     return result;  
13 }  
14 // windows([1,2,3,4], 2) -> [[1,2], [2,3], [3,4]]
```

Sostituzione senza modificare la lista originale

```
1 public static <T> List<T> replace(  
2     List<T> list,  
3     Predicate<T> toReplace,  
4     T replacement) {  
5     List<T> result = new ArrayList<>();  
6     for (T element : list) {  
7         result.add(toReplace.test(element) ? replacement : element);  
8     }  
9     return result;  
10 }
```

Estrarre una porzione come copia indipendente

```
1 public static <T> List<T> extract(  
2     List<T> list,  
3     int from,  
4     int to) {  
5     if (from < 0 || from > to || to > list.size()) {  
6         throw new IndexOutOfBoundsException();  
7     }  
8     return new ArrayList<>(list.subList(from, to));  
9 }  
10 // from incluso, to escluso; il risultato non è una vista
```

13.4 ! Griglie: Position come chiave di Set e Map

Forma ricorrente dell'Esercizio 2

Usa `Set<Position>` per celle occupate o cliccate e `Map<Position,V>` quando ogni cella possiede un valore. Un `record` è una buona chiave perché genera `equals` e `hashCode` coerenti ed è immutabile.

Validazione e aggiornamento atomico del click

```

1 record Position(int x, int y) { }
2
3 private final Set<Position> marked = new HashSet<>();
4 private final int size = 5;
5 private final int target = 4;
6 private boolean over;
7
8 boolean hit(Position position) {
9     if (position == null || over
10         || !inGrid(position)
11         || marked.contains(position)) {
12         return false;
13     }
14     marked.add(position);
15     over = marked.size() >= target;
16     return true;
17 }
18
19 boolean inGrid(Position position) {
20     return position.x() >= 0 && position.x() < size
21         && position.y() >= 0 && position.y() < size;
22 }

```

! Ordine del metodo hit

Prima valida coordinate, duplicati e stato di fine; soltanto dopo modifica la collezione. Infine aggiorna la condizione di vittoria. Cambiare quest'ordine produce spesso click contati due volte o stato corrotto.

14 Tabella di scelta rapida

Tipo	Duplicati	Ordinamento	Quando usarlo
ArrayList	sì	inserimento	lista standard e accesso frequente tramite indice
ArrayDeque	sì	estremità	coda/deque/pila; scelta standard per operazioni alle estremità
LinkedList	sì	inserimento	quando servono contemporaneamente i contratti List e Deque
HashSet	no	nessuno	elementi unici e ricerca rapida
LinkedHashSet	no	inserimento	elementi unici mantenendo l'ordine di inserimento
TreeSet	no	naturale/Comparator	elementi unici, ordinati e navigabili
HashMap	chiavi uniche	nessuno	associazioni chiave-valore rapide
LinkedHashMap	chiavi uniche	inserimento	mappa rapida con ordine deterministico
TreeMap	chiavi uniche	per chiave	associazioni con chiavi ordinate e navigabili

Regola pratica

Come scelta iniziale usa `ArrayList`, `HashSet` e `HashMap`. `TreeSet` e `TreeMap` ordinano secondo l'ordine naturale o un `Comparator`. Per conservare l'ordine di inserimento usa `LinkedHashSet` o `LinkedHashMap`. Scegli `ArrayDeque` quando il lavoro avviene soprattutto alle estremità.

15 ! Errori tipici d'esame

- Copiare dal PDF codice contenente trattini Unicode al posto di `->` o `-`.
- Chiamare `size()` su `Iterable`: il metodo appartiene a `Collection`.
- Cercare `get(i)` su un `Set`: i set non hanno indici.
- Scrivere `list.remove(1)` pensando di rimuovere il valore 1: su una lista di interi viene rimosso l'elemento di indice 1.
- Credere che `HashSet` o `HashMap` mantengano l'ordine di inserimento.
- Confondere `TreeSet/TreeMap` con l'ordine di inserimento: `Tree*` ordina, `LinkedHash*` conserva l'inserimento.
- Dimenticare `LinkedHashSet` quando servono elementi unici in ordine di inserimento.
- Modificare campi usati da `equals/hashCode` mentre l'oggetto è un elemento di `HashSet` o una chiave di `HashMap`.
- Usare un tipo non confrontabile dentro `TreeSet` senza fornire un `Comparator`.
- Dimenticare che `Arrays.asList()` produce una lista a dimensione fissa e che un array primitivo diventa un solo elemento.
- Confondere una vista come `subList` con una copia indipendente.
- Usare `Stream.toList()` e poi provare a modificare la lista.
- Usare `List.copyOf(...)` pensando di ottenere una `ArrayList` modificabile.
- Dimenticare che `List.of(...)` e `List.copyOf(...)` non accettano `null`.
- Usare `Map.entry(...)` con una chiave o un valore `null`.
- Confondere `Map` con `Collection`: le viste sono `keySet()`, `values()` ed `entrySet()`.
- Usare soltanto `get(key) == null` quando la mappa può contenere valori `null`.
- Usare `binarySearch` su dati non ordinati con lo stesso criterio della ricerca.
- Riutilizzare uno `Stream` dopo un'operazione terminale: lo stream è già stato consumato.
- Confondere `map` con `flatMap`: il primo trasforma, il secondo trasforma e appiattisce.
- Usare `Collectors.toMap` senza funzione di fusione quando possono comparire chiavi duplicate.
- Chiamare `Optional.get()` senza verificarne il contenuto: preferire `orElse`, `orElseGet` o `ifPresent`.
- Usare `orElse(expensive())` pensando che il valore alternativo venga calcolato soltanto quando serve.
- Scrivere una finestra con limiti errati oppure non gestire `size <= 0`.
- Aggiornare lo stato di una griglia prima di avere validato il click.
- Credere che `import java.util.*` importi anche `Collectors`: serve `java.util.stream.Collectors`.

! Quattro template da sapere a memoria

```
map.put(key, map.getOrDefault(key, 0) + 1) per contare; map.computeIfAbsent(key, k -> new
ArrayList<>()).add(value) per raggruppare; return Optional.empty() quando una ricerca fallisce; new
ArrayList<>(source) per ottenere una copia modificabile.
```